Project Report

Subset Sums Puzzles

William Hanson

1 INTRODUCTION

This project proposes to solve a numerical puzzle that is an application of the Subset Sums Problem (SSP). Three different techniques: Brute Force, Dynamic Programming, and Backtracking will be used to solve the puzzle. The performance and limitations of these three approaches will be compared. The report will conclude with observations, lessons learned, and recommendations for further research.

1.2 Problem Description

The specific problem is a number sums puzzle. This puzzle has a grid of numbers and required sums for each row and column. The object is to remove all numbers from the grid that are not part of the subset either by row or by column. A simple version is shown in Figure 1. This is essentially an interlocking set of Subset Sum puzzles. This project will focus on finding integer solutions for these puzzles.

	6	3	21	19
19	2	4	8	9
9	8	7	6	3
11	4	8	1	7
10	2	3	7	7

Problem

	6	3	21	19
19	2		8	9
9			6	3
11	4			7
10		3	7	

Solution

Table 1: Example Problem

1.2 Problem Importance

The general problem of subset sums, along with the closely related "knapsack problem," is important in both theoretical and real-world applications. These include areas like cryptography [1], financial auditing, resource allocation, and most recently, quantum computing [2]. The addition of the second problem axis allows us to use different techniques, as it now becomes a Constraint Satisfaction Problem (CSP) along with being a subset sums problem. This will allow us to show important strengths and limitations of both SSP and CSP approaches.

2 BRIEF REVIEW OF PREVIOUS LITERATURE

While there are many articles covering the general idea of subset sum problems (SSPs), the associated Knapsack problem, and Constraint Satisfaction Problems (CSPs), searching the literature did not turn up any articles covering the specific number sums puzzle used for this project. However, there are several types of puzzles that have similar features, including Latin Squares, Magic Squares, Sudoku, and Nomograms.

However, we shall start with the basic SSP. This is a well-known NP-complete problem in computer science [3]. The basic idea is that given n integers x, $1 \leq x \leq k$, and a target b, we want to find the if there is a subset D such that

$$x_1 + x_2 + x_3 + \cdots + x_n = b$$

As Russel and Norvig note, an NP-complete problem is a way to understand if a given problem is tractable or not. NP complete problems are generally considered to be intractable, although that has not been proven.[4] The SSP is essentially a subset of the Knapsack Problem, which similarly aims to maximize the number of items of different sizes and weights that can fit

into a knapsack, was the basis of an early attempt to create an unbreakable encryption algorithm via the Merkle-Hellman algorithm.[5] However, this algorithm was shown to be breakable in a variety of ways, as summarized by Odlyzko.[1]

One important conclusion that can be drawn is that, while the overall problem itself may be NP-hard, it can be solved in pseudo-polynomial time. A general approach involves partitioning the problem into subsets, solving those subsets, and then building up a solution from the solved subsets.[6] One key approach, Dynamic Programming, was first developed by Richard Bellman at RAND in the early 1950s and has been a powerful tool ever since. Interestingly, Bellman's original motivation was to solve the problem of how to optimally direct weapons to targets to maximize expected damage, a multi-stage stochastic problem that I worked on extensively in the late 1980s. One of the issues with this problem is that it greatly increases, or "explodes" in dimensionality as the numbers of weapons and targets increase. As such, one of Bellman's important conceptual breakthroughs is that such problems have common features. In particular, once a single decision is made, this collapses the set of parameters of the next decision stage, and, that the past history of the system is unimportant in determining future actions.[7] In practice, once we make the original decision, this simplifies the universe of future decisions.

Applied to the SSP, this means that once a subset is determined, this forecloses other potential subsets, and the problem simplifies itself as more subsets are determined. This essentially partitions the problem, and this partition allows the SSP to be solved in pseudo-polynomial time, $O(mn^2)$, where m is the target sum and n is the size of the array of numbers.[8][9]

Shifting focus, since the structure of the puzzle essentially creates a set of row and column constraints, it falls into the class of Constraint Satisfaction Problems (CSPs) and therefore a backtracking approach with forward checking should be a viable method of solving these puzzles. Backtracking is essentially a variant of depth-first search on the puzzle grid, but because the search space for a CSP is commutative, meaning that it does not matter which order the variables are chosen, this greatly reduces the search space. Even so, the problem space of n assignments with d possible values is still d^n.[10] This is still a significant issue, which will be seen in the subsequent analysis as puzzle size increases.

At the same time, an attractive feature of backtracking algorithms is that they will always solve CSPs – even though they may do it very slowly.[11] This, of course, assumes that a solution exists. One important facet in the literature is that Backtracking does best when it is paired with methods to check the consistency of assignments. This can take either the form of Forward Checking or Arc Consistency, along with other methods.[12][13]

An additional method of attacking CSPs is through Boolean Satisfiability. This method models the CSP as a Boolean function via propositional logic and then attempts to satisfy that function while using either a backtracking or branch and bound approach to detect assignments that fail to meet constraints and then either backtrack or prune those incorrect assignments. This also reduces the problem space as it goes.[14][15]

As mentioned above, while the particular number puzzle being used is not directly addressed in the literature, similar puzzles, such as Magic Squares[16], Latin Squares[17], Sudoku[18], and Nomograms[19] have been studied. This report will not discuss these in detail, other than to note that the basic mechanisms for solving these puzzles generally fall into the same algorithmic approaches as SSPs or CSPs, along with heuristics associated with the specific puzzle structure.

3 SOLUTION APPROACHES AND RATIONALE

This project will take three approaches to solve these puzzle problems: Brute Force (BF), Dynamic Programming (DP), and backtracking along with Boolean satisfiability (BT).

3.1 Brute Force

As a baseline, the first approach is to simply use brute force. The problem grid is decomposed into its respective rows and columns. Proceeding row-wise, all possible permutations of each row are calculated. Since numbers can repeat within rows and columns, the order of the combinations is important. For example, if we have the row [2, 4, 4, 2] then both [2, 4] and [4, 2] are distinct combinations. Accordingly, we have to consider all permutation sizes from 1 to n for a row of n numbers. In general, the number of permutations can be calculated by:

$$\sum_{k=1}^{n} \frac{n!}{(n-k)!}$$

Obviously, this number can get very large. For an 8x8 puzzle, which is the largest puzzle of this type found in online games, this works out to 109,600 permutations for each row and column, while for a 20x20 puzzle, which is the largest we will consider in this analysis, this works out to 6.6×10^{18} permutations for each row and column.

ALGORITHM 1: BRUTE FORCE

```
def brute_force(numbers, target):
    def create_subsets(numbers, target):
        for i in len(numbers):
            create permutations of number (1,i)
            if sum_permutation == target:
                permutation = valid
                valid_subsets.append(permutation)
        return valid_subsets
    def find_invalid_numbers(valid_subsets, numbers):
        for i in len(numbers):
            if numbers(i) not in valid_subsets:
                return invalid
    def find_required_numbers(valid_subsets, numbers):
        for i in len(numbers):
            if numbers(i) in all valid_subsets
                return required
    for each row and column:
        create_subsets
        find_invalid_numbers
        find_required numbers
    if all numbers == valid or invalid:
        return solution
    else restart with first row
```

The Brute Force (BF) implementation first takes the puzzle row-wise, calculating all the possible permutations and then only keeping the ones that sum to the row target. For example, if the row is [2, 3, 4, 2] and the target is 6, that would cut the number of valid subsets down to two, [2, 4] and [4, 2].

Once the valid subsets are determined, then the algorithm performs two additional checks: the first for the numbers in the original list that are not part of any valid subset, and the second for any number that is in all valid subsets. From our previous list of [2, 3, 4, 2], the number 3 is not in any valid subset and is therefore eliminated from further considerations, and the number 4 is required in any valid subset, and therefore must be part of the solution.

To keep track of the status of numbers while maintaining the original puzzle structure, a mask is used to indicate whether the number is eliminated from further consideration (-1), required (1), or still undecided (0). Accordingly, the row mask for our example row would start at [0, 0, 0, 0] and, after the first pass, would be [0, -1, 1, 0].

Once all the rows are considered, then the algorithm considers the puzzle column-wise. Here, the mask pays immediate dividends since it greatly limits the number of allowable column combinations, both by eliminating some numbers from consideration and requiring others to be in the list of potential solutions. Further, when a number is mandatory, the respective target is decremented by that number. Returning to our first row [2, 3, 4, 2], where 3 is eliminated and 4 is mandatory, when we consider the 3rd column, its target is decremented by 4. If that column is [4, 7, 3, 2] and its target is also 6, then by subtracting our mandatory 4 from the target, that leaves 2 as the revised target, which eliminates 7 and 3, leaving only 2 remaining, and the column is solved with the numbers 4 and 2.

The process continues for all the columns, at which point the overall puzzle mask is checked to see if all the spots are either 1 or -1. If that is true, then the puzzle is solved. Otherwise, the algorithm makes another iteration through the rows and columns until it is either complete or the algorithm stalls due to being unable to find the correct subsets. Since each pass simplifies the overall problem, the BF method was quite efficient, being able to solve an 8x8 puzzle in 2-3 iterations, and a 20x20 puzzle in at most 7 iterations. Mean run times ranged from about 1.5 milliseconds for an 8x8 puzzle and 17 milliseconds for a 20x20 puzzle. The speed of calculating permutations was aided by using the Python *itertools* library with its optimized functions.[20]

3.2 Dynamic Programming

Another approach that immediately suggested itself was Dynamic Programming (DP). Here, we identify the process of finding subsets of the rows and columns that sum to the target numbers as a series of subproblems and then solve those subproblems, continuing until a solution is found. One of the key advantages of DP over the previous BF method is that rather than defining all the possible subsets and then testing them, DP determines only the subsets that sum to the target.

The DP algorithm takes a different approach to the problem of finding subsets. It decomposes the problem into smaller subsets and then uses those solved subsets to determine if those will sum to the target value. The important part here is that the algorithm keeps track of which subsets work and which do not work. It does this by using a Boolean table to track whether elements of the list sum to a given value.

Rather than writing out the algorithm, it is more understandable via a simple example, let's say that our list is [4, 2, 3] and our target (T) is 7. We start by creating a table with dimensions n+1 and T+1. The top row numbers are the different sums [0 – 7] that can be reached, with 7 being our target. The left column is the number of elements [0 – n] that can be used. At each spot, we ask two questions: Can the sum at the top be reached by either including or excluding that element? If either is true, then we mark the spot as True, and otherwise False.

Starting with the first column, is the target sum (0) reachable by including or excluding the numbers in the set. We can always get zero by excluding all numbers, so that column is True, similarly, when looking at the column sums at the top, we cannot get those sums using zero elements, so the rest of that row is False, giving us:

	0	1	2	3	4	5	6	7
0	T	F	F	F	F	F	F	F
1	T							
2	T							
3	T							

Now, taking the first element (4), we move along the 2nd (1) row and ask if we can get the sum at the top using the element 4 by subtracting 4 from the top sum, if that result is zero, then the spot is true, which occurs under the 4 column. We also check to see if we can reach that sum by excluding 4, which is always false for this row, resulting in:

	0	1	2	3	4	5	6	7
0	T	F	F	F	F	F	F	F
1	T	F	F	F	**T**	F	F	F
2	T							
3	T							

Moving to our second element (2), we again ask if we can reach the desired sum by either including or excluding 2 along with the previous element (4). Here, we can reach the sum of 2 by including 2 and excluding 4 (the F in the row above indicates that 4 is excluded). We can also reach the sum of 4 by excluding 2, and we can reach the sum of 6 by including both elements. We know this because subtracting 2 from the target of 6 leaves 4. Then we go up a row to the previous element and back two, which puts us in the 4 column. That entry is True, so (2,6) is also True. This gives us:

	0	1	2	3	4	5	6	7
0	T	F	F	F	F	F	F	F
1	T	F	F	F	**T**	F	F	F
2	T	F	**T**	F	**T**	F	**T**	F
3	T							

This points out the key workings of the algorithm. We've been decomposing the overall problem into sub-problems, and now we know that we can get a sum of either 0, 2, 4, or 6 by either including or excluding elements of our set. Further, the algorithm doesn't have to re-check to see if a combination is valid. It simply decrements the target and then checks the resulting column in the previous row to see if it is True or False. While this difference is trivial for the small example, it makes a large difference in larger sets of numbers.

Taking our last element (3), we again go down the row, asking if the target sum at the top can be reached by either including or excluding 3. This gives the final result of:

	0	1	2	3	4	5	6	7
0	T	F	F	F	F	F	F	F
1	T	F	F	F	**T**	F	F	F
2	T	F	**T**	F	**T**	F	**T**	F
3	T	F	**T**	**T**	F	**T**	**T**	**T**

Accordingly, the target sum of 7 can be reached by including the elements, 4 and 3. While this description sounds cumbersome, in practice it is very efficient at solving these types of problems. As pointed out above, the subset sum problem was originally thought to be NP-hard, but the Dynamic Programming approach showed that it could be solved in pseudo-polynomial time[8].

Once the valid subsets are in hand, the algorithm continues row- and column-wise through the puzzle, much like the BF algorithm. Similarly, for each row or column, it eliminates non-feasible values, includes required values, and then reevaluates that reduced set. Again, a [-1, 0, 1] mask was used to keep track of everything and the stopping logic was the same.

Because of this commonality, runtimes were similar to the BF algorithm, but were consistently faster than it because of its efficiency at finding valid subsets. DP solution speed ranged from a mean of about 1.4 to 12 milliseconds for puzzles ranging from 8 to 20 rows and columns.

3.3 A CSP approach: Backtracking

Since the intersection of the row and column must satisfy both row and column subsets, this requirement allows us to quickly eliminate invalid numbers, simplifying the puzzle. The previous two approaches did this, first when the row was checked, and then when the column was checked. Here, we have a chance to potentially speed this up by checking both at once. At the same time, this implies that the puzzle is also a CSP, and as such a backtracking approach with forward checking should be a workable approach. However, doing this ended up being considerably more difficult than originally anticipated.

The backtracking approach is a fairly straightforward application of recursion.

ALGORITHM 2: BACKTRACKING

```
def backtracking(number, target):
    pick next number from list
    def backtrack(number, current_sum, current_subset):
        if current_sum == target
            return current_subset                    #solution found
        if current_sum > target:
            return None
        # Explore including current_number
        current_sum += number
        current_subset.append(number)
        result = backtrack(next_number, current_sum, current_subset)
        # Explore excluding current_number
        current_subset.pop()                         # Remove last number
        result = backtrack(current_sum, current_subset)
        if result:
            return result
        else:
            return None                              # No solution this branch
```

Rather than doing forward checking when a new value is selected, I opted to do it beforehand by making a single row and column pass using the techniques described above while using backtracking to find the subsets rather than either of the previous methods. This significantly cut down on the size of the CSP problem, greatly cutting down runtimes for solving puzzles, and gave the backtracking algorithm a "forward-checked" puzzle right from the stop. However, this had some unintended consequences. One of which is that the backtracking algorithm often got very deep into the solution before it realized that one of the first picks was invalid. This led to the algorithm constantly thrashing back and forth across the entire puzzle making it painfully slow. How slow? The first runs for an 8x8 puzzle came in at over 35 minutes (vs. around 1.5 ms for the previous two algorithms).

While the algorithm worked, it worked glacially. To get puzzle solutions down a workable time, I searched for an optimized package to help with the number crunching and increase speed. This led to the Python *constraint* library.[21] However, this was not much better, only improving performance to around 25 minutes. The journey ended with Google's *OR-tools* library.[22] This got runtimes down to workable 20 ms.

The use of OR-tools required me to recast the backtracking problem as a set of Boolean constraints. In this case, every spot ($x_{i,j}$) on the puzzle grid had a Boolean (T/F) variable – the backtracking algorithm then started by setting the first available spot to True, then checked the row and column constraints, then tried the next available spot, and so on, recursing back if a constraint was violated. This was the same basic approach as my original implementation, only much faster. I should stress that using the external package was a convenience that allowed me to speed up an already-working algorithm. When that approach was working, it was time to start evaluating the relative performance of all three algorithms.

3.4 Program Structure

The final program took a modular structure. Since the logic for solving the puzzle took three general steps: finding subsets, validating subsets, and then stepping through the puzzle, separate modules were created for each step, with similar functions in each module tailored to their respective algorithm. This is shown in Figure 1.

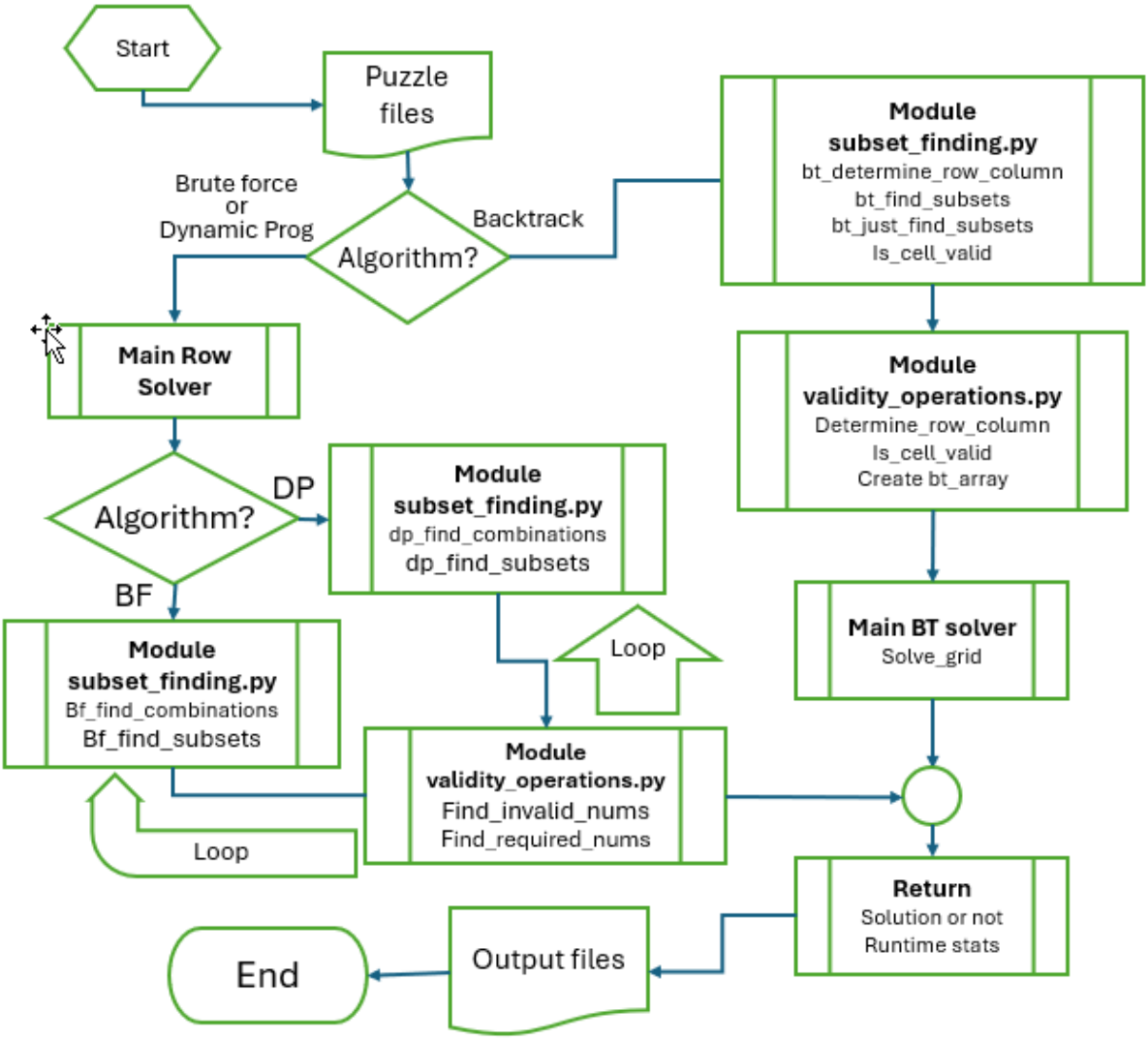


Figure 1: Program Flowchart

As can be seen on the flowchart, the program starts by loading the puzzle file. Then, depending on which algorithm is chosen, it either goes to solve the program row- and column-wise via the main row solver logic for BT and DP algorithms, or to the BT solvers. For the row solver, it loops through finding and validating subsets as the program checks the rows and columns until the puzzle is either solved or the process stalls due to the program not being able to find a unique solution.

The BT branch first does a single pass of finding and validating row and column subsets, leaving it with a “pre-filtered” grid which significantly reduces the puzzle size. That is then passed to the BT grid solver.

Once complete, the program passes the solution status and runtime statistics to an output file. This was all embedded in a loop, which would sequentially load each puzzle to the program, solve it, and then move to the next program.

The overall system also included separate programs for creating and checking puzzle files, loading and parsing the puzzle files, and a set of programs to analyze and plot the results.

4 EVALUATION OF APPROACHES

As touched on earlier, when looking at runtimes, both the Brute Force and the Dynamic Programming methods provided comparable runtimes, while the Backtracking algorithm came in starting at 10 times longer and got worse as the size of the puzzles increased. This would seem to indicate that these two row-based approaches were the clear winners. However, the BT approach was much more robust than the other two in terms of its ability to solve a wide variety of puzzles.

4.1 Puzzle Construction

It was clear from the beginning that if we intended to evaluate the performance of these algorithms against larger grids, then it would be necessary to "reverse engineer" the existing examples to understand how they were built. With that in hand, we could then apply those rules to create increasingly large puzzles as needed. The largest available puzzles were an 8x8 grid. It was assumed the general approach to producing a puzzle was to randomly place a subset of integers on the grid. Then, the row and column sums for that subset would form the sum targets. This would guarantee a solvable puzzle. Then the remainder of the grid could be filled in with random numbers. The key factors were the percentage of the grid covered by the solution, the distribution of numbers in the solution, and the overall distribution of numbers on the grid. After hand-solving 100 of these puzzles, it was determined that the solution covered an average of 36% of the grid (~33 spots on an 8x8 grid) and that both solution and overall distribution were generally uniform across the domain (1, 9).

4.1.1 Ability of Algorithms to Solve Puzzles

Applied to an 8x8 grid, the set of values determined above worked for creating other solvable puzzles, but once 10x10 and larger puzzles were being created, it was discovered that most could not be solved by either the BF or DP algorithm, although they could be solved by the BT algorithm. Some experimentation revealed that decreasing the percentage of squares that were part of the solution from 36% to 10-20% helped, but still left many puzzles unsolvable for the BF or DP algorithms. For example, holding the percentage of squares part of the solution constant at 20% and increasing the puzzle size showed that the larger the puzzle was, the worse the BF and DP algorithms performed, as shown in Figure 2, below.

	8x8	**10x10**	**12x12**	**15x15**	**20x20**
% Solved by Brute Force	100%	83%	73%	23%	0%
% Solved by Dynamic Programming	100%	83%	73%	23%	0%
% Solved by Backtracking	100%	100%	100%	100%	100%

Table 2: Percentage Solved by Algorithm vs. Puzzle Size (n = 100 randomly generated puzzles)

4.1.2 Problem Solvability

Investigating this problem revealed that the issue was not that the created problems were unsolvable, but that the created problems had too many solutions. The DP and BT algorithms were created to solve puzzles with a single solution. However, the process of creating larger puzzles tended to create puzzles with multiple solutions, and the larger the puzzle got, the higher the percentage of puzzles with multiple solutions.

To illustrate, consider this simple example puzzle below. The four outside corner numbers are all "2" and each of their respective rows and columns only requires one "2" to meet its row and column target. Intuitively, we can select either (1,1) and (4,4) or (1,4) and (4,1) to solve the puzzle, but neither the DP or BF algorithm can either uniquely rule any of those numbers in or out of the puzzle, so the solver stalls at that point and exits without a solution.

	10	x	x	5
3	2	1	5	2
x	7	x	x	3
x	1	x	x	4
14	2	6	6	2

Figure 2: Mutually Exclusive Solutions

The BT algorithm, however, does not try to rule those numbers in or out. It instead would select (for example) the 2 at (1,1) for inclusion, traverse to (1,4), rule that out and then rule out (4,1) before eventually including (4,4) and completing the puzzle. As the size of created puzzles grew, the probability that mutually exclusive puzzle solutions would exist grew even faster, resulting in the condition where less than 0.001% of created 20x20 puzzles had a single valid solution.

4.1.3 Final Problem Creation

For smaller puzzles, it was just a process of trial and error to find enough random puzzles to give a reasonable (200 – 500) number of solvable puzzles to evaluate algorithm performance. As mentioned above, the exception was the 20x20 puzzles. After generating 2,500 puzzles yielded only 2 that were solvable, a different approach was needed, which was to use existing puzzles to create "synthetic" puzzles by swapping single rows and columns. Intuitively, swapping a pair of rows or columns (along with their respective targets) does not change the validity of the solution. Swapping rows does not affect the column sums, nor does it affect the row sums since the targets are swapped with the rows. Using this approach, it was possible to expand the number of solvable 20x20 puzzles from 2 to 722 puzzles. It could be argued that such an approach creates essentially identical puzzles, and that is true. At the same time, in terms of how the algorithms approached the puzzle, they were enough different to require completely new solutions for each pass. At most, this would somewhat increase the already significant jump in runtimes between 15x15 and 20x20 puzzles.

These difficulties made it so the practical limit for this project was creating 20x20 puzzles. However, these were significantly larger than the largest available pre-made puzzles, which are a maximum of 8x8.

4.2 Comparison of algorithms

As the figure and table below shows, while runtimes for both BF and DP algorithms were roughly comparable, runtimes for the BT approach started out about 10 times longer than the other two, and that difference got worse the larger the puzzle got. This made doing comparison charts difficult as it was impossible to meaningfully show all three on the same graph. Accordingly, the remainder of the charts will show the results of the BF and DP algorithms together and the BT results separately.

4.2.1 Comparison of Runtimes

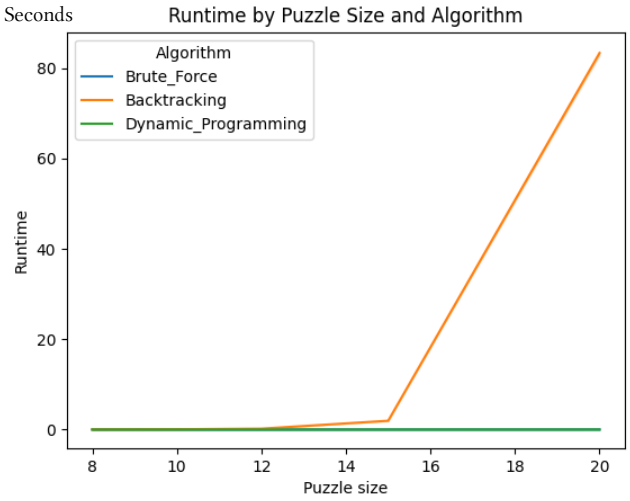


Figure 3: Runtimes (seconds) by Puzzle Size: All Algorithms

	Puzzle Size				
Algorithm	**8x8**	**10x10**	**12x12**	**15x15**	**20x20**
Backtracking	0.0199	0.0573	0.1993	1.9488	83.3753
Brute Force	0.0015	0.0034	0.0037	0.0059	0.0170
Dynamic Programming	0.0014	0.0031	0.0031	0.0051	0.0120

Table 3: Mean runtime in seconds

Looking at a boxplot of the BF and DP results showed that they were fairly tightly clustered, as shown below.

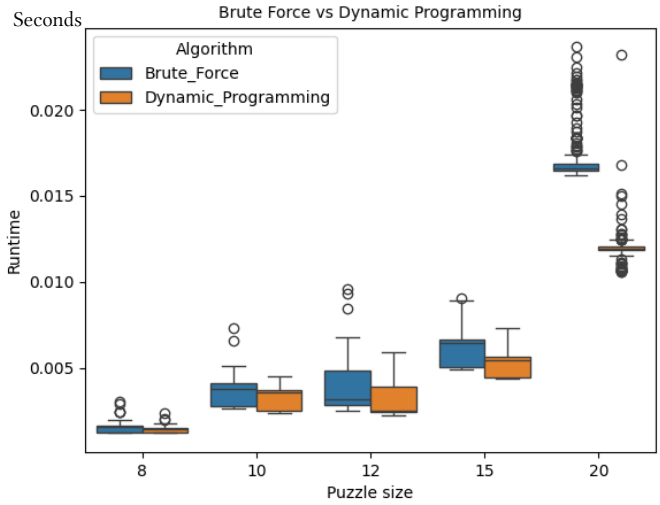


Figure 4: Brute Force and Dynamic Programming Runtime Distribution

One thing that became clear was that the runtimes for each algorithm did not appear to be normally distributed. As an example, looking at the histogram of runtimes revealed a multi-modal distribution, as exemplified by the results for 10x10

puzzles shown below. The various peaks correspond to how many iterations were needed to solve the puzzle. To confirm this impression, the Shapiro-Wilk test for normality was run, with results consistently confirming that the runtimes were definitely not normally distributed. In all cases, the test statistic showed a p-value < 0.00001, indicating a non-normal distribution.

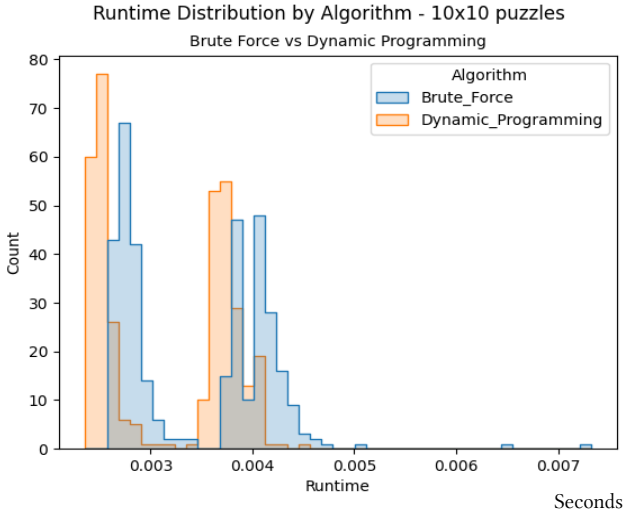


Figure 5: Runtime Histogram for 10x10 puzzles

Given this, normal methods of determining statistical difference, such as a paired t-test, could not be used. Since the boxplots seem suggestive that the DP algorithm was faster than BF, a non-parametric test for difference, a Permutation test, was performed to confirm if the results were statistically distinct.[23] The results confirmed that the differences between the two means were statistically significant in all cases, with a minimum p-value of 0.0056 in the 8x8 puzzle case. As shown in the box-plots above in Figure 4, that speed advantage for DP only grew as the puzzles got larger.

The distribution of runtimes for the BT algorithm showed much the same behavior, as shown in Figure 6, below. We can see that the runtimes took a huge jump from 15x15 to 20x20 puzzles, from a mean of about 2 seconds for a 15x15 puzzle to over 83 seconds for a 20x20 puzzle.

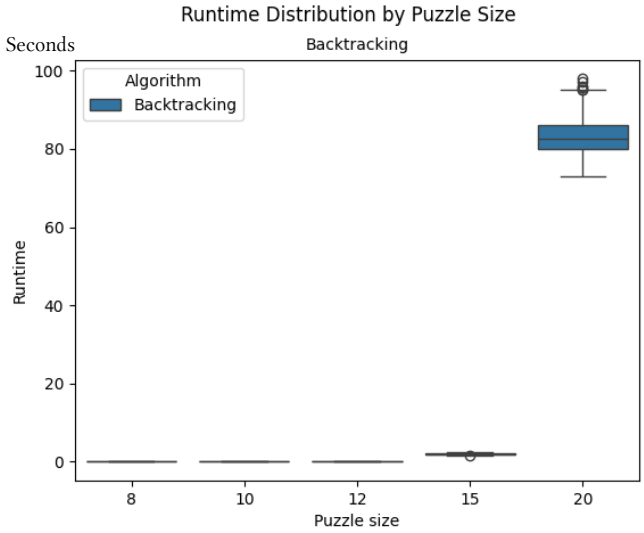


Figure 6: Backtracking runtimes by puzzle size

This exponential increase is a direct result of the algorithm. Recall that we defined our BT puzzle grid as a set of Boolean variables. This meant that the size of the problem space increased by a factor of 2^n, where n is the total number of spots on the puzzle grid. This caused the size of the 20x20 puzzle space to scale to a factor of 2^{400}, and even with a highly-optimized algorithm this greatly increased problem size would quickly test the limits of a desktop computer.

One thing that partially ameliorated the effect of this exponential increase was the "pre-filtering" that was performed before turning the problem over to the full backtracking algorithm. This was designed to replace "forward checking" by eliminating the numbers that clearly did not meet the row and column constraints. In practice, this also significantly reduced the number of Boolean variables, from a mean of about 1/3 of the puzzle to a maximum of 1/2 of the puzzle, as shown in the figure below, which illustrates this for a 15x15 puzzle.

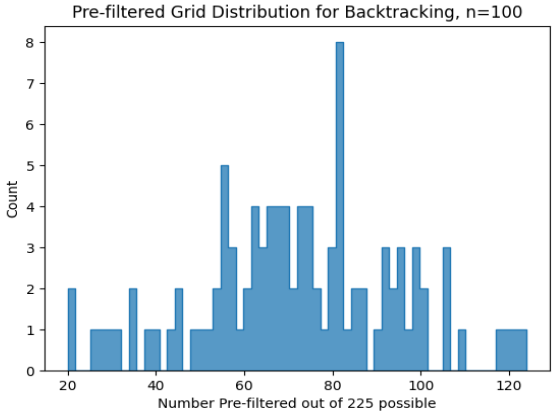


Figure 7: Number of "Pre-filtered" spots for a 15x15 puzzle

4.2.2 Regression/Prediction of Runtime Results

Since the distribution of runtimes was not normally distributed, and the previous line and boxplots showed significant non-linear response as puzzle size increased, approaches like Linear Regression are clearly not useful as predictive tools, either for puzzle sizes in between those used for this analysis (e.g., 9x9 or 17x17) or those beyond the upper end of 20x20. However, given that the BT algorithm increased in size exponentially with the puzzle size and DP algorithms were shown to be able to solve subset sums problems in polynomial time, this section will do exponential and polynomial curve-fitting via the *numpy polyfit* tool [24] to see if those algorithms follow either exponential or polynomial time growth.

Looking at first the BT approach, which should clearly be exponential, running an exponential curve fit gave an initially impressive-looking R^2 of 0.94, which indicates that the fitted equation explained 94% of the variance in runtimes. However, this is misleading as the fitted curve had a Mean Squared Error (MSE) of 66.05. This implies that the average error was a little more than 8 seconds, which for a 20x20 puzzle was about a 10% error. This, alongside the plot in Figure 8 below means that we must take this curve prediction as illustrative rather than definitive.

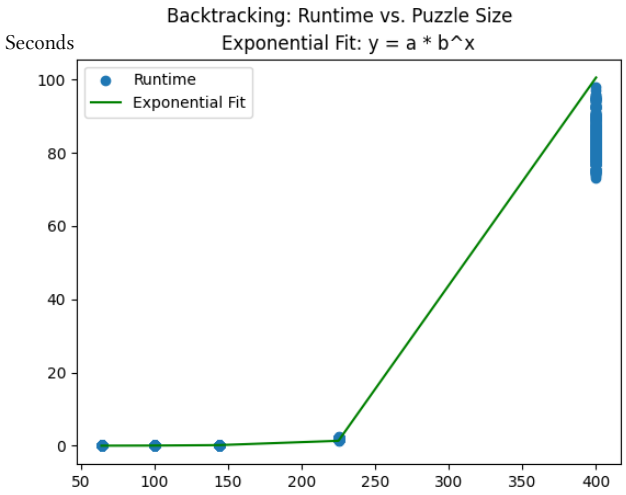


*Figure 8: Exponential fit for Backtracking (Fitted model: y = 5.376e-03 * 1.025^x)*

Turning to the DP algorithm, we expected a polynomial increase in runtimes as puzzle size increased, so a polynomial fit was used. This also resulted in a high R^2 of 0.94, but this time, the MSE was < 0.0001, showing a very good fit on average even though the data was not normally distributed. This is reflected in the plot in Figure 9, below.

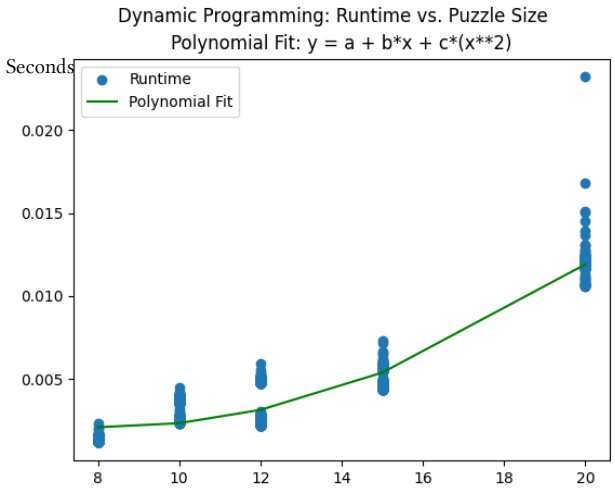


Figure 9: Polynomial fit for Dynamic Programming
*(Fitted model: y = 6.685e-03 + -1.128e-03*x + 6.946e-05*x^2)*

Checking the BF algorithm against both exponential and polynomial fits returned similar results with R^2 values of about 0.94 and MSE < 0.0001 in both cases. Over the range of puzzles studied, even a linear fit showed a good R^2 value of 0.85. However, as mentioned above, these results were not normally distributed, so while these results indicate reasonably good curve fits, they cannot be taken as definitive. The exponential fit for the BF algorithm is shown in Figure 10.

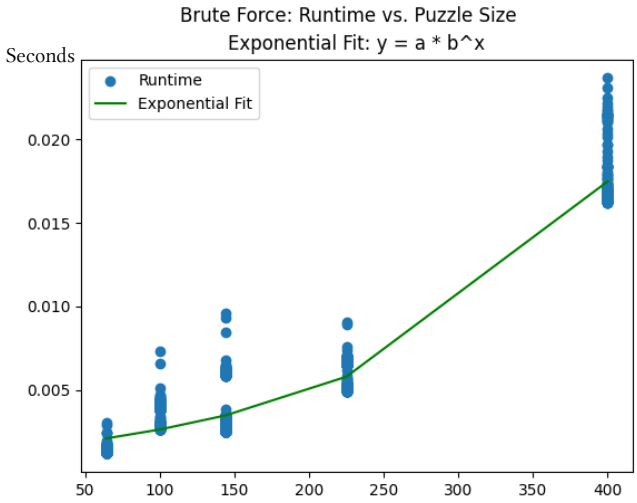


*Figure 10: Exponential fit for Brute Force (Fitted model: y = 1.400e-03 * 1.006^x)*

5 LESSONS LEARNED

There were several lessons learned in the development and execution of this project. These mostly involved the increasing size and difficulty of the puzzle solution space. The fact that problems would get more difficult as they got larger was expected, but the magnitude of the increase was unexpected, particularly for the Backtracking (BT) algorithm. Intuitively, I originally thought that the BT algorithm was going to be the quickest at solving puzzles because it was not necessary to go through the process of computing the subsets in the same way as the Brute Force (BF) or the Dynamic Programming (DP) algorithms. This lack of computational overhead should have given it an advantage.

However, this did not work out in practice. A big part of the issue is that the method of using Boolean variables in the backtracking algorithm greatly increased the size of the problem. As discussed earlier, given a puzzle size of *n* Boolean variables, this resulted in a state space of 2^n, growing the size of the puzzle from 2^{64} for an 8x8 puzzle to 2^{400} for the 20x20 puzzles. Being able to "pre-filter" the puzzles, as discussed in Section 4.2 significantly cut down on that size problems, but its impact was very uneven, ranging from reducing the initial puzzle size by about 10% to almost 50%, with a corresponding change in runtimes needed to solve the puzzle.

This illustrated the tradeoff between "pre-filtering" the puzzle and forward-checking as the algorithm progressed. Doing the first cut down the effective initial puzzle size significantly. However, giving the BT algorithm a puzzle where most of the spots potentially were valid caused the algorithm to thrash significantly as it would go deep into the puzzle before running into an impasse and then having to backtrack all the way to the beginning. Whether changing to forward-checking at each step would have ameliorated this issue is unknown, as there was not enough time to revise the algorithm before the deadline.

Another important lesson that was learned was that partitioning the problem is a very effective method. While the slow performance of the backtracking algorithm was unexpected, the fact that the Brute Force algorithm did not react similarly was equally unexpected. As Figure 3 shows, it wasn't until the puzzles got larger than 15x15 that the BF algorithm started to significantly diverge from the DP algorithm. As discussed before, treating the problem as a set of Boolean variables for each spot on the puzzle grid caused the problem space to expand at the rate of 2^n. Partitioning the problem row and column-wise changed the problem to a series of much smaller problems, each only 2^m, where m = sqrt(n). For an 8x8 puzzle, partitioning the space into 8 rows and 8 columns decreased the initial problem space from 2^{64} to $16 * 2^8$. Further, as rows were solved, the column problems quickly became simpler and vice versa. Most puzzles were solved in less than 4 iterations, and even the 20x20 puzzles took a maximum of 7 iterations.

At the same time, while partitioning the puzzle resulted in significant advantages, the disadvantage was that these approaches had no way to deal with situations as described above in Section 4.1.2. This caused larger puzzles to become increasingly unsolvable for both the BF and DP algorithms, while the BT algorithm could find a solution when the other algorithms could not. The most difficult published puzzles are 8x8 grids, which points out the increasing difficulty of creating larger puzzles that still

have a unique solution.[25] As discussed in Section 4.1, this necessitated a trial and error approach, but that essentially became untenable at 20x20 grids or larger.

6 CONCLUSION

This project set out to perform two tasks: Create two algorithms to solve the number puzzles and extend those algorithms to solve puzzles larger than the 8x8 puzzles available online. Both of these goals were exceeded. Three algorithms were created, and through a combination of reverse engineering and testing, puzzles with more than six times the numbers in the largest commercial "hard" puzzles (400 vs 64) were successfully created and solved.

While each of the algorithms developed during this project could easily solve the most difficult commercial puzzles, the limits of all three approaches became abundantly clear as the puzzles got larger. All three approaches essentially hit the "wall" at 20x20 puzzles, at least on desktop hardware. Using parallel processing and faster hardware would most likely extend the size of puzzles that could be solved via the Backtracking (BT) algorithm, but simply throwing more computing resources at the problem has its limits as well. Further, the difficulty of constructing larger puzzles with unique solutions limits the usability of either the Brute Force (BF) or Dynamic Programming (DP) algorithms. Accordingly, this problem cannot be declared "solved" by any of the three approaches.

Even staying within those limitations, determining the "best" algorithm depends on the metric. In terms of speed, DP was the clear winner, as it was faster at solving puzzles of any size than the others. However, the DP approach needs to have a puzzle with a unique solution to work and, therefore, was not robust at solving larger puzzles.

Here, BT was the clear winner, as it was able to solve any puzzle with at least one solution. Unfortunately, it was very slow. Even with an 8x8 puzzle, it was, on average, 10x slower than the other two approaches, and that relative difference rapidly increased as the puzzle size increased, growing to almost 5,000x slower for 20x20 puzzles.

To extend this research, an important challenge would be to combine the robustness of the BT solver with the speed of the DP algorithm. While using a BT algorithm with the same row and column partitioning suggests itself, that approach would most likely suffer from the same fundamental problem that BF and DP had, which was the inability to deal with problems with multiple solutions because they only looked at the problem by row or column.

However, the opposite approach could be used, which would be to have BT effectively act as a "tie breaker" when the DP algorithm reached an impasse. Briefly stated, the DP algorithm would run until it was unable to make more progress, then a BT algorithm would take over to get past that point, and then hand the problem back to the DP algorithm, alternating approaches as needed until the problem was solved.

Another approach would apply the insight that partitioning can make the problem effectively much smaller. While it isn't possible to cleanly partition the number grid other than by row or column, one could use a heuristic to create sub-grids and assign numerical targets for those sub-grids. This might be as simple as (for example) dividing a 12x12 puzzle into four 3x3 subgrids and making the targets ½ the value of the original targets, solving those 4 problems and then testing the entire grid based on those solved sub-problems. We can think of different ways to sub-divide the puzzle grid and different ways to assign targets, and this would be an interesting problem for longer-term research.

A third potential approach would be to relax the integer solution requirement and treat the puzzle as a Linear Programming (LP) problem. After the LP problem was solved, then we could search for "close" integer solutions. Similarly, casting the problem as an n-dimensional geometric problem, while initially challenging, might provide a way of solving very large problems of this nature.

A fourth approach appears most promising. Instead of using individual numbers for the Backtracking approach, we could instead represent the valid subsets for the rows and columns as vectors. Then, we could select the first row vector, test it against the column vectors, and if no column vectors could be found to "fit" the row vector, then discard that row vector from consideration and try the next vector. If the row 1 vector is valid against the column vectors, then add the row 2 vector and check it against the set of column vectors that agreed with the row 1 vector, check those as before, then either backtrack to a new row 2 vector and continue until the puzzle is solved. I was trying to implement this approach but was unable to make it work before the project deadline. Even so, I am confident this would greatly increase the speed of the Backtracking approach, plus it would gain from the ability to subdivide the problem in this fashion.

While this project cannot claim to have solved this problem, at the same time, we have gained more insight into the issue and see the limitations of our original thinking. More important, we also see potential avenues for improvement. This points out that, just as in life, even after navigating the necessary tradeoffs, “solving” a problem merely advances us to a new set of opportunities, problems, and tradeoffs. The journey continues.

REFERENCES

[1] Andrew Odlyzklo. 1990. The Rise and Fall of Knapsack Cryptosystems. In Proceedings of Symposia in Applied Mathematics, Vol. 42.

[2] David Biesner, Thore Gerlach, Rafet Sifa, Christian Bauckhage, and Bernd Kliem. 2022. Solving Subset Sum Problems using Quantum Inspired Optimization Algorithms with Applications in Auditing and Financial Data Analysis. In Proceedings of the IEEE International Conference on Machine Learning Applications.

[3] Jeffrey.C. Lagarias & Andrew Odlyzko. 1985. Solving Low-Density Subset Sum Problems. Journal of the ACM. 32, 1 (January 1985), 229-246

[4] Stuart Russel & Peter Norvig. 2022. Artificial Intelligence: A Modern Approach, 4th Ed. Pearson. p. 27

[5] Ralph Merkle & M. Hellman. 1978. Hiding Information and Signatures in Trapdoor Knapsacks. IEEE Transactions on Information Theory. 24, 5 (September 1978), 525-530.

[6] Jon Kleinberg & Èva Tardos. 2006. Algorithm Design. Pearson. p. 266.

[7] Richard Bellman & E. Stanley Lee. 1984. History and Development of Dynamic Programming. IEEE Control Systems Magazine. 4, 4 (November 1984), 24-28.

[8] Thomas O'Neil. 2013. An Empirical Study of Algorithms for the Subset Sum Problem. Proceedings for the 46th Midwest Instruction and Computing Symposium (MICS 2013) University of Wisconsin - La Crosse. April 19-20 2013.

[9] Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein. 2022. Introduction to Algorithms, 4th. Ed. The MIT Press.

[10] Russel & Norvig. AIMA, p 175

[11] Vipin Kumar. 1992. Algorithms for Constraint-Satisfaction Problems: A Survey. AI Magazine. 13, 1 (Spring 1992), 32-14.

[12] Peter van Beek. 2006. Backtracking Search Algorithms. In Rossi, van Beek, & Walsh (eds.) Handbook of Constraint Programming. Elsevier. 83-116.

[13] Hans Keller, Ulrich Pferschy, David Pisinger. 2004. Knapsack Problems. Springer-Verlag.

[14] Sharad Malik and Lintao Zhang. 2009. Boolean Satisfiability: From Theoretical Hardness to Practical Success. Communications of the ACM. 52, 8 (August 2009), 76-82.

[15] Russel & Norvig. AIMA, p. 255

[16] Anany Levitan & Maria Levitin. 2011. Algorithmic Puzzles. Oxford University Press.

[17] Mark Jacobsen & Peter Matthews. 1996. Generating Uniformly Distributed Random Latin Squares. Journal of Combinatorial Designs. 4, 6 (January 1996) 405-437.

[18] Helmut Simonis. 2005. Sudoku as a Constraint Problem. In CP Workshop on modeling and reformulating Constraint Satisfaction Problems, Sitges Spain. 12. 13-27.

[19] I.-Chen Wu, Der-John Sun, Lung-Ping Chen, Kan-Yueh Chen, Ching-Hua Kuo, Hao-Hua Kang, & Hung-Hsuan Lin. 2013. An Efficient Approach to Solving Nonograms. 2013. IEEE Transactions on Computational Intelligence and AI in Games. 5, 3 (September 2013) 251-264.

[20] Python Itertools Library. https://docs.python.org/3/library/itertools.html

[21] PyPi Constraint 1.4.0. https://pypi.org/project/python-constraint/

[22] Google OR-Tools. CP-SAT. https://developers.google.com/optimization/cp

[23] S. Siegel and N.J. Castellan. 1988. Nonparametric Statistics for the Behavioral Sciences, 2nd ed. McGraw-Hill.

[24] Numpy Polyfit. https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html

[25] Easybrain, Ltd. 2024. Number Sums – Numbers Game. Apple App Store.